INDEX

Exp.No:1 Date:	Introduction to azure boards and azure devops

AIM:

To explore the Azure DevOps environment and demonstrate the creation, management, and tracking of work items (Epics, Features, and User Stories) using Azure Boards.

PROCEDURE:

1. Introduction to Azure DevOps

Definition: Azure DevOps is a Microsoft Cloud service that provides an end-to-end toolchain for developing and deploying software. It facilitates DevOps (Development and Operations) practices by integrating people, processes, and technology to deliver continuous value to users.

Core Components

- Azure Boards: Work tracking with Kanban, Sprints, and Backlogs.
- Azure Repos: Private Git repositories for source control.
- Azure Pipelines: CI/CD (Continuous Integration and Continuous Deployment) services to build and release applications.
- Azure Test Plans: Integrated manual and exploratory testing tools.
- Azure Artifacts: Integrated package management (Maven, NuGet, Python).

---

2. Azure Boards: Project Management

Azure Boards is the specific service used for planning, tracking, and discussing work across the entire development team.

Key Work Item Hierarchy

1. Epic: High-level business initiatives (e.g., "Mobile App Development").
2. Feature: Specific objectives within an Epic (e.g., "User Login").
3. User Story / Task: Smallest unit of work that can be completed in a sprint.

4. Bug: Used to track defects or issues in the code. Visual Tracking Tools

- Kanban Board: A visual interface to manage the flow of work. It helps identify bottlenecks by visualizing work in columns (e.g., *New*, *Active*, *Resolved*, *Closed*).
- Backlogs: A prioritized list of work items. The Product Backlog shows the longterm plan, while the Sprint Backlog shows what will be done in the current cycle.
- Sprints: Time-boxed iterations (usually 2 weeks) where the team commits to finishing a set of work items.

RESULT:

Thus, the concepts of Azure boards & azure devops are studied successfully

Exp.No:2 Date:	Creation of azure account

AIM:

To create and configure a Microsoft Azure account and set up an Azure DevOps organization for managing software development projects.

PROCEDURE:

Step 1: Accessing Azure Portal

1. Open your web browser and navigate to the official Azure website (azure.microsoft.com).
2. Click on the "Start free" button.
3. Sign in with your Microsoft account credentials. If you do not have one, select "Create one!" to register a new email.

Step 2: Identity Verification

1. Enter your basic profile information (Country, Name, and Phone Number).
2. Identity Verification by Phone: Provide your mobile number to receive a verification code via SMS or call. Enter the code to proceed.
3. Identity Verification by Card: Provide credit/debit card details.

Step 3: Accessing the Dashboard

1. Once verified, click "Sign up" to agree to the subscription agreement.
2. You will be redirected to the Azure Portal Dashboard, which serves as the central hub for all cloud services.

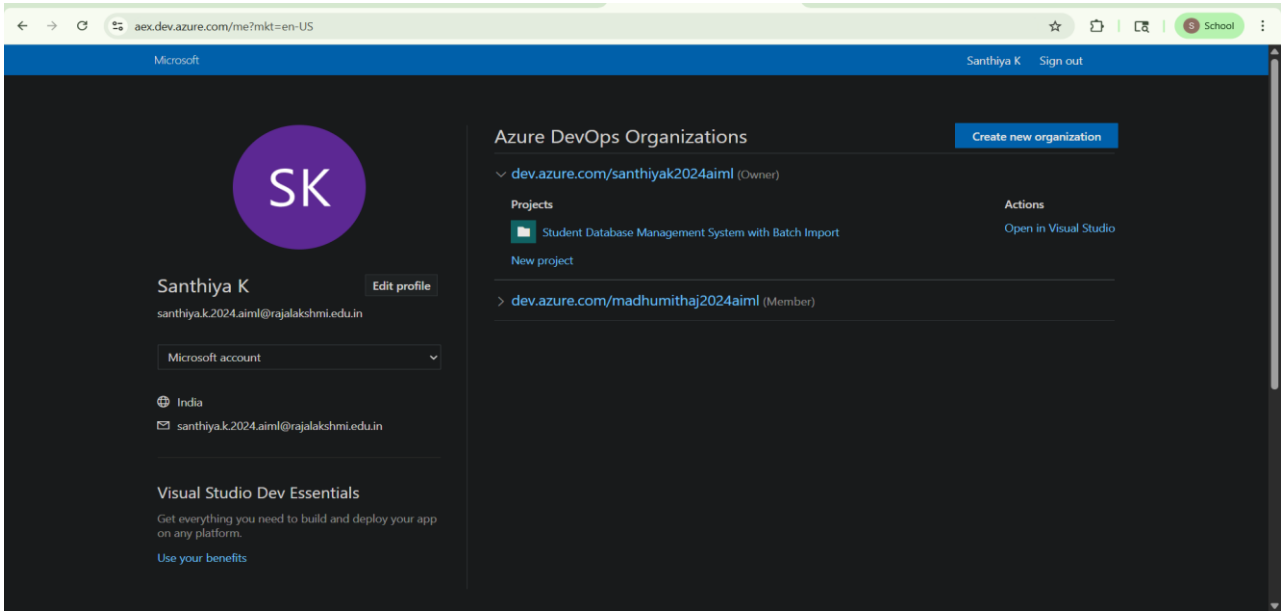
Step 4: Setting up Azure DevOps Organization

1. In the Azure Portal search bar, type "Azure DevOps" and select it.
2. Click on "My Azure DevOps Organizations."
3. Click "Create new organization."
4. Choose a unique name for your organization and select the server region closest to your location (e.g., South India).
5. Complete the CAPTCHA and click "Continue."

Step 5: Project Creation

1. Once the organization is ready, you will be prompted to "Create a project to get started."
2. Enter a Project Name and set the Visibility (Public or Private).
3. Click "Create project." Your Azure Boards and other DevOps services are now active.

OUTPUT:



RESULT:

The Microsoft Azure account was successfully created, and the Azure DevOps environment was initialized with a new project, ready for work item tracking and development.

Exp.No:3 Date:	Creating Epics, Features and User Stories for Project Planning

AIM:

To demonstrate the creation of an Azure DevOps Organization and establish a work item hierarchy by defining Epics, Features, User Stories, and Tasks within a project.

PROCEDURE:

Step 1: Creating the Organization

1. Log in to dev.azure.com using your Microsoft account.
2. Click on New Organization in the left-hand panel.
3. Enter a unique name for the organization and select the preferred hosting region.
4. Click Continue to initialize the DevOps environment.

Step 2: Project Initialization

1. On the "Create a project" screen, enter a Project Name.
2. Set the visibility to Private.
3. Under Advanced, ensure the Work Item Process is set to Agile (or Scrum, depending on your requirement).
4. Click Create Project.

Step 3: Creating an Epic

1. Navigate to Boards > Work Items in the sidebar.
2. Click + New Work Item and select Epic.
3. Title: Enter a high-level goal (e.g., *Launch Mobile Banking App*).
4. Assign a priority and click Save.

Step 4: Creating Features (Child of Epic)

1. Inside the Epic, go to the Related Work section.
2. Click Add link > New item.
3. Set Link type to "Child" and Work item type to Feature.
4. Title: Enter a functional module (e.g., *User Authentication System*).

5. Click OK and then Save.

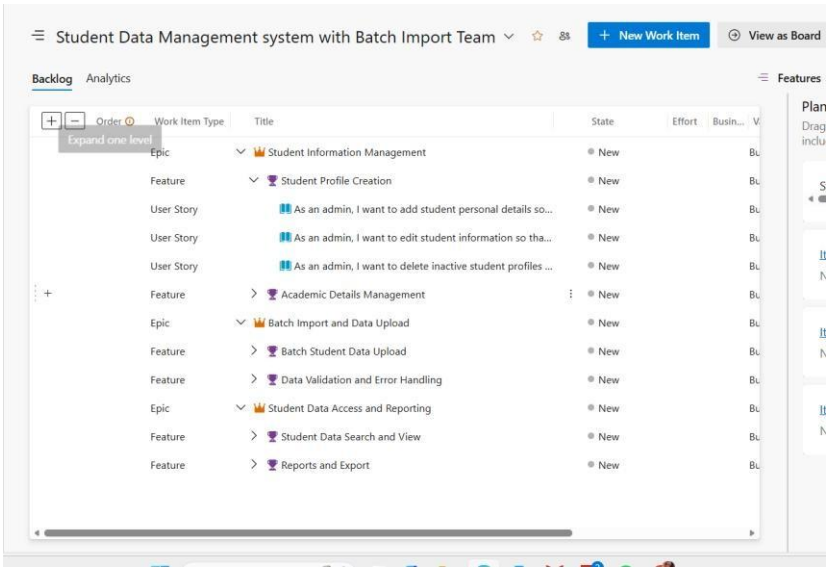
Step 5: Creating User Stories (Child of Feature)

1. Open the Feature created in the previous step.
2. Under Related Work, click Add link > New item.
3. Set Work item type to User Story.
4. Title: Enter a user-centric requirement (e.g., *As a user, I want to log in using my email*).
5. Click Save.

Step 6: Creating Tasks (Child of User Story)

1. Open the User Story.
2. Add a "Child" link and select Task.
3. Title: Enter a specific technical action (e.g., *Design Login UI* or *Configure Database Connection*).
4. Click Save & Close.

OUTPUT:



RESULT:

The Azure DevOps organization was successfully configured, and a structured hierarchy of work items was created, enabling clear traceability from high-level business goals down to individual developer tasks

Exp.No:4 Date:	Sprint Planning

AIM:

To implement Agile project management by organizing the *Student Data Management System* requirements into a time-boxed Sprint.

PROCEDURE:

Technical Background: Sprint Planning is a collaborative event in Scrum where the team defines what can be delivered in the upcoming sprint and how that work will be achieved. It transforms the "What" (Product Backlog) into the "How" (Sprint Backlog).

Step 1: Organization Setup: Log in to Azure DevOps and select your project. Navigate to Boards > Sprints.

Step 2: Defining Iterations: Set the Sprint duration (e.g., May 1st to May 14th).

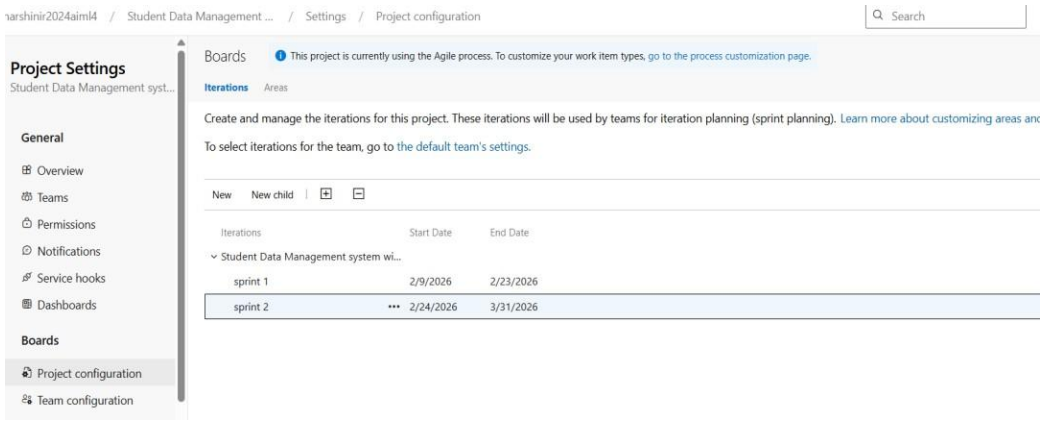
Step 3: Backlog Grooming: Review the "Student Data Management" backlog. Identify high-priority items like *CSV Batch Import Logic* and *Database Schema Setup*.

Step 4: Capacity Planning: Under the Capacity tab, assign hours for each team member (e.g., 6 hours/day for development, 2 hours/day for testing).

Step 5: Task Mapping: Drag User Stories into the current sprint. Decompose the "Batch Import" story into granular tasks:

- *Task 1:* Design File Upload UI.
- *Task 2:* Implement CSV Parsing Library.
- *Task 3:* Write SQL Bulk Insert Script.

OUTPUT:



RESULT:

A fully populated Sprint Backlog with a defined goal, ensuring the team has a clear roadmap for the next two weeks.

Exp.No:5 Date:	Poker Estimation

AIM:

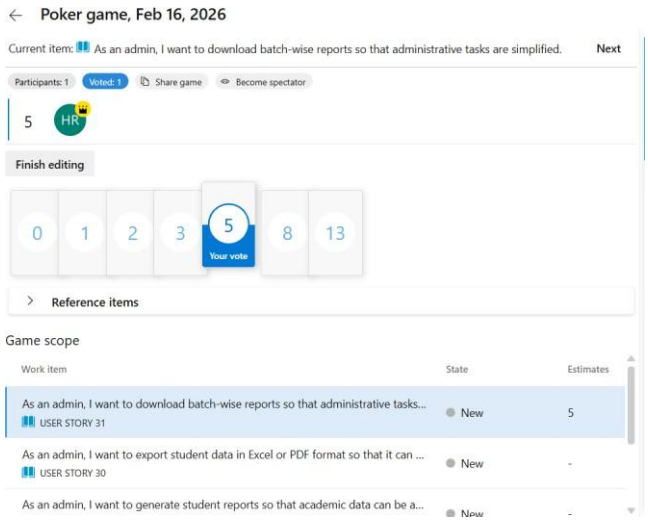
To perform collaborative effort estimation for the "Batch Import" module using the Planning Poker technique.

PROCEDURE:

Technical Background: Planning Poker uses the Fibonacci sequence to represent uncertainty and complexity. It prevents "Social Loafing" and "Anchoring" by requiring all team members to vote simultaneously.

1. Setup: Launch the Planning Poker extension in Azure Boards or use a web-based poker tool.
2. Story Review: The Product Owner explains the requirements for the *Batch Import* feature, including data validation and error logging for failed rows.
3. Round 1 Voting: Each member selects a card (1, 2, 3, 5, 8, 13).
4. Discrepancy Discussion: If a developer votes '2' and a tester votes '8', the team discusses the testing complexity of large datasets.
5. Final Consensus: After a second round of voting, a consensus estimate (e.g., 5 Story Points) is reached.
6. Recording: The Story Point value is saved in the Effort field of the Work Item.

OUTPUT:



RESULT:

A consensus-based estimation that provides a realistic view of the project's complexity and team velocity.

Exp.No:6 Date:	Designing Class and Sequence Diagrams for Project Architecture

AIM:

To design the structural (static) and behavioral (dynamic) architecture of the Student Data Management System using UML modeling to ensure a robust technical blueprint for the "Batch Import" module.

PROCEDURE:

1. Requirement Extraction: Analyze the project specifications to identify the core entities involved in the Student Data Management System.
2. Defining Attributes: For each class, determine the necessary data fields:
    - Student: studentID (PK), name, batchCode, email, department.
    - BatchProcessor: fileName, uploadDate, status.
3. Defining Methods: Determine the logic each class must execute:
    - BatchProcessor: parseCSV(), validateData(), handleErrors().
    - DatabaseHandler: dbConnect(), bulkInsert().
4. Establishing Relationships: Define how the classes connect:
    - Composition: A Batch contains multiple Student objects.
    - Association: The BatchProcessor sends data to the DatabaseHandler.
5. Notation Application: Draw the classes in a three-tier box format. Use + for public access and - for private access.
6. Review: Ensure the "Batch Import" service is properly linked to both the user interface and the backend database.
7. Sequence Diagram (Dynamic Behavior) Theory:

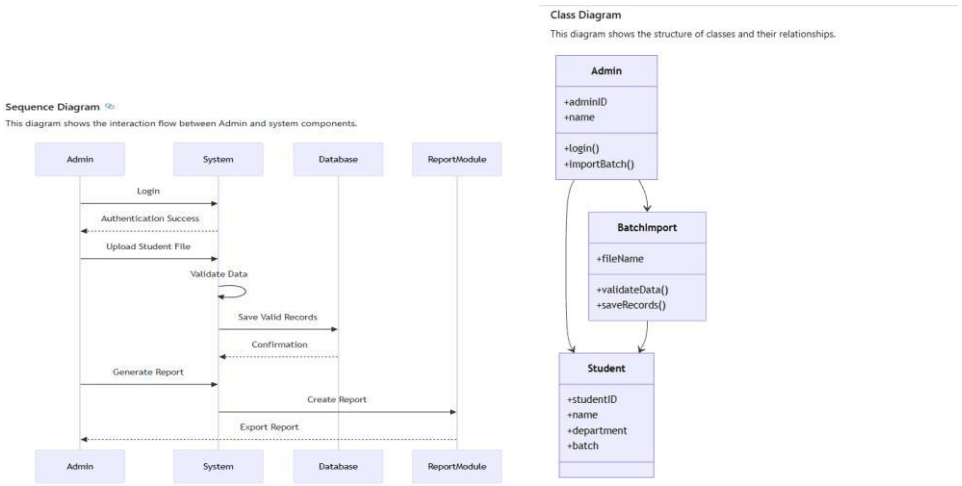
A Sequence Diagram is an interaction diagram that shows how processes operate with one another and in what order. It is a visualization of the "Batch Import" use case over time.

1. Actor & Object Identification: Define the participants involved in the import workflow:
    - Actor: System Admin.
    - Lifelines: UploadUI, ImportController, FileParser, and Database.
2. Message Flow Mapping:

- The Admin sends a message to UploadUI by selecting a CSV file and clicking "Upload."
- UploadUI passes the file object to the ImportController.

3. Action Execution:
   - The ImportController sends a request to FileParser to validateData(). ○ The FileParser iterates through the records and returns a success or failure code.
4. Implementation of Logic Fragments:
   - Create an Alt Fragment (Alternative).
   - Condition [Valid]: The ImportController calls bulkInsert() on the Database. ○ Condition [Invalid]: The ImportController returns an errorResponse() to the UI.
5. Completion: Draw return messages (dashed arrows) showing the final confirmation (e.g., "500 Records Imported Successfully") reaching back to the Admin.

OUTPUT:



RESULT:

The structural architecture (Class Diagram) and the behavioral interactions (Sequence Diagram) were successfully designed. This dual-modeling approach ensures that the "Batch Import" module is technically sound, handling data ingestion logic and database persistence in a chronologically organized and modular fashion

Exp.No:7 Date:	Designing Use-Case and Activity Diagrams for Project Architecture

AIM:

To visualize the functional requirements and the internal procedural logic of the Batch Import feature.

PROCEDURE:

Procedure for User case Diagram

Step 1: Draw the System Boundary (a large rectangle) and label it "Student Data Management System."

- Step 2: Identify the Primary Actor. Place the "Admin" stick figure on the left side of the boundary.
- Step 3: Identify Secondary Actors. Place the "Database Server" and "SMTP Server" on the right side of the boundary.
- Step 4: Define Core Use Cases. Inside the boundary, draw ovals for Login, Batch Upload, View Logs, and Download Report.
- Step 5: Draw Associations. Connect the Admin to all use cases and connect the "Batch Upload" use case to the Database Server.
- Step 6: Implement Relationships. Use <<include>> to connect Batch Upload to Login, indicating that a user must be authenticated to perform an upload.

Procedure for Activity Diagram

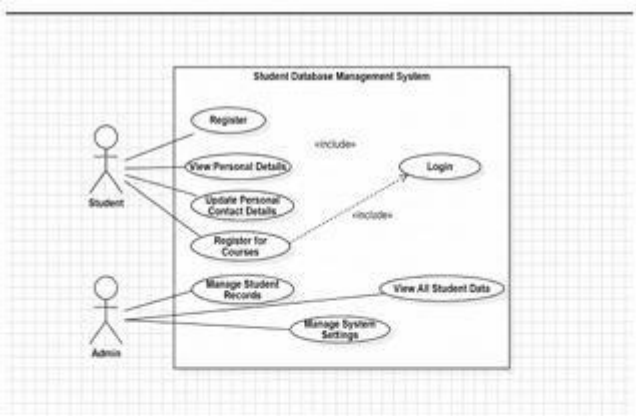
- Step 1: Place the Initial Node (a solid black circle) at the top of the diagram.
- Step 2: Create the Start Action. Draw a rounded rectangle labeled Step 1: Admin selects CSV file and clicks Upload.
- Step 3: Insert a Decision Diamond. Labeled as Step 2: Is file extension .csv?.
- Step 4: Define the Error Path (Path A). If "No," move to an action labeled Step 3a: Display Error Notification and lead to the final node.
- Step 5: Define the Processing Path (Path B). If "Yes," move to an action labeled Step 3b: Parse CSV Data.
- Step 6: Insert a Fork Symbol (a thick horizontal line). This indicates the system will now perform two actions simultaneously.
- Step 7: Map Parallel Actions.

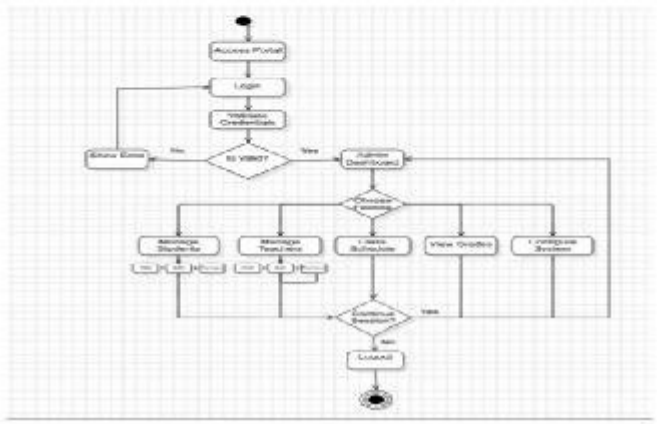
- Sub-step A: Step 4a: Insert Valid Records to Database. ○

  Sub-step B: Step 4b: Log Invalid Rows to Error Report.

- Step 8: Insert a Join Symbol. Use a thick horizontal line to merge the two parallel flows back into a single path.
- Step 9: Define the Termination. Lead to an action labeled Step 5: Display Import Summary and terminate at the Final Node (a bullseye symbol).

OUTPUT:





RESULT:

The functional requirements and logical pathways were successfully mapped, providing a clear visual guide for both user interactions and back-end logic branching.

Exp.No:8 Date:	Testing – Test Plans and Test Cases

AIM:

To establish a rigorous Quality Assurance (QA) protocol using Azure Test Plans for the Student Data Management System.

PROCEDURE:

Step 1: Navigate to the Azure Test Plans module in the DevOps portal and click "New Test Plan." Title it "Student Management System - Release 1.0."

Step 2: Create a Test Suite. Right-click the plan and select "New Static Suite" for Global functions and a "Requirement-based Suite" for the Import module.

Step 3: Click New Test Case and enter a unique ID and Title (e.g., TC_101: Verify Batch Import with Null Grade Values).

Step 4: Define Pre-conditions. State that the Admin must be logged in and a sample CSV file must be ready. Step 5: Enter Test Steps in the grid:

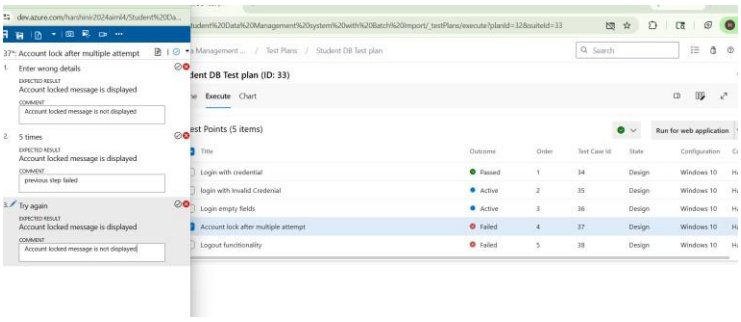
- Action 1: Select "Batch Import" from the menu.
- Action 2: Upload a CSV where the "Grade" column is empty for some students.
- Action 3: Click "Process."

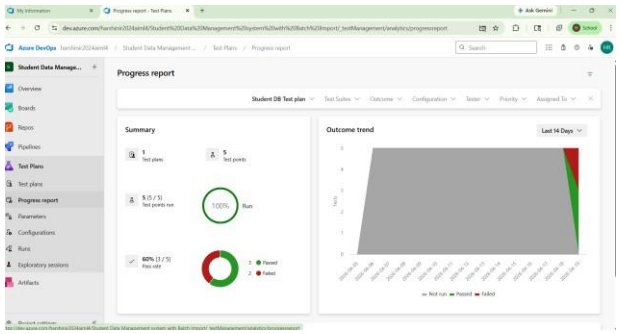
Step 6: Define Expected Results. State: "System should flag the empty grades as warnings but allow the import to proceed for other valid fields."

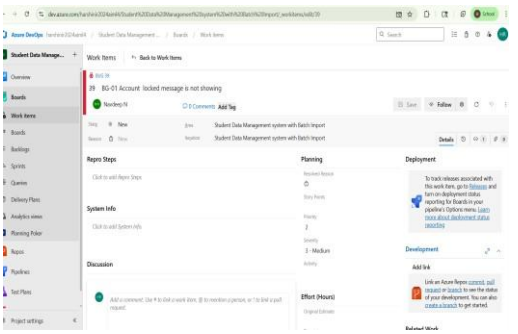
Step 7: Establish Traceability. Use the "Related Work" tab to link the test case to the original "Batch Import" User Story in Azure Boards.

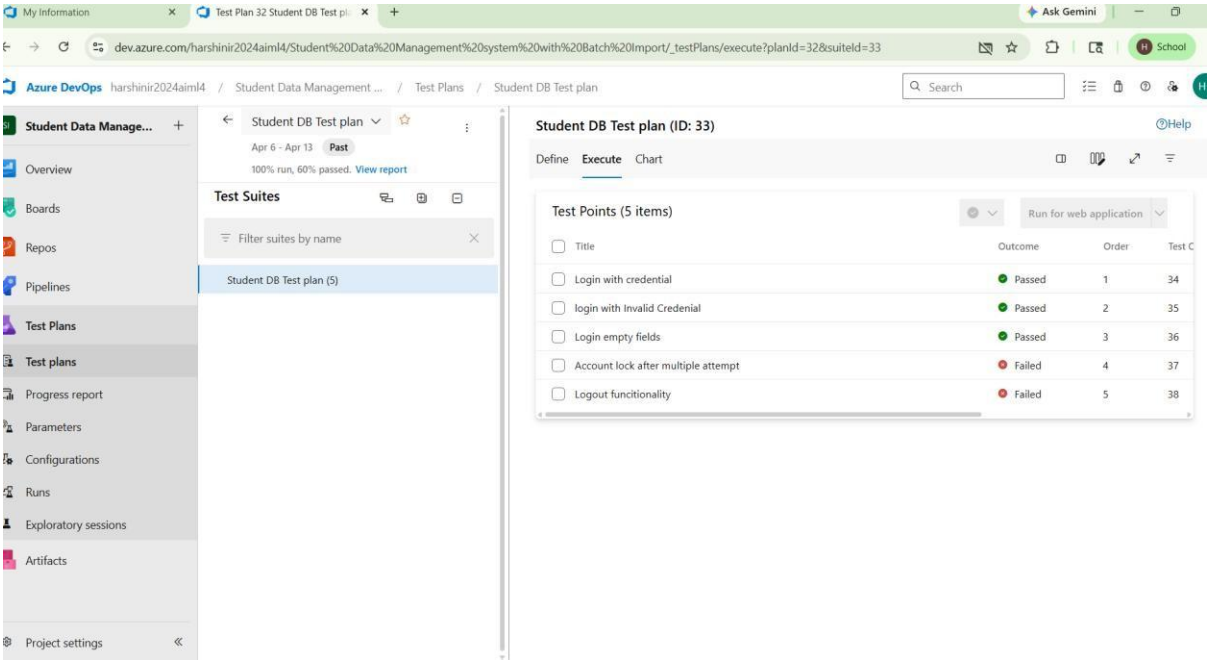
Step 8: Execute the Test. Open the "Test Runner," perform the actions in the live system, and mark each step as "Pass" or "Fail." Use the "Capture Screenshot" tool to attach evidence of any bugs found.

OUTPUT:









RESULT:

A comprehensive QA framework was initialized, ensuring that every functional requirement is backed by a repeatable and traceable test scenario

Exp.No:9 Date:	Creation of login page

AIM:

To develop a secure, responsive, and fully integrated authentication gateway for the Student Data Management System.

PROCEDURE:

- Step 1: Environment & Directory Structure Initialization ○ Open VS Code and create a root project folder.
    - Inside the root, create three sub-directories: /public/css, /public/js, and /public/assets. ○ Place your existing login.html in the /public folder to maintain a clean separation between static assets and server logic.
- Step 2: Refining the HTML5 Semantic Structure
    - Open login.html. Ensure the <form> element has a unique ID (e.g., id="loginForm") for JavaScript targeting. ○ Add a with id="errorMessage" immediately above the form to display validation alerts.
    - Link the external stylesheet and script file using:
        - <link rel="stylesheet" href="css/style.css">
        - <script src="js/script.js" defer></script>
- Step 3: Implementing Modern UI with CSS3 Flexbox ○ Open style.css. Define a global reset to remove default margins and padding.
    - Container Styling: Apply display: flex; and align-items: center; to the body to perfectly center the login card vertically and horizontally. ○ Card Aesthetics: Apply a white background to the form container, add a box-shadow: 0 4px 8px rgba(0,0,0,0.1);, and set border-radius: 10px; for a soft, modern look. ○ Interactive Elements: Add :hover and :focus states for the input fields and the submit button to improve user experience (UX).

Step 4: Client-Side Logic & Validation (JavaScript)

- Open script.js. Use document.getElementById('loginForm').addEventListener('submit', ...) to intercept the click event.
- Prevention: Immediately call event.preventDefault() to stop the page from refreshing.
- Validation Logic: Write an if statement to check if username.value or password.value are empty strings.
- Visual Feedback: If empty, manipulate the DOM to change the errorMessage text to "Fields cannot be blank" and set its color to red.

•

Step 5: API Integration with Async/Await Fetch ○ Inside the same script, if validation passes, initialize a fetch() request.

- Set the method to 'POST' and the headers to 'Content-Type': 'application/json'.
- Use JSON.stringify() to convert the form data into a JSON object for transmission to the backend endpoint /api/login.
- Wrap the call in a try...catch block to handle network errors or server downtime gracefully.

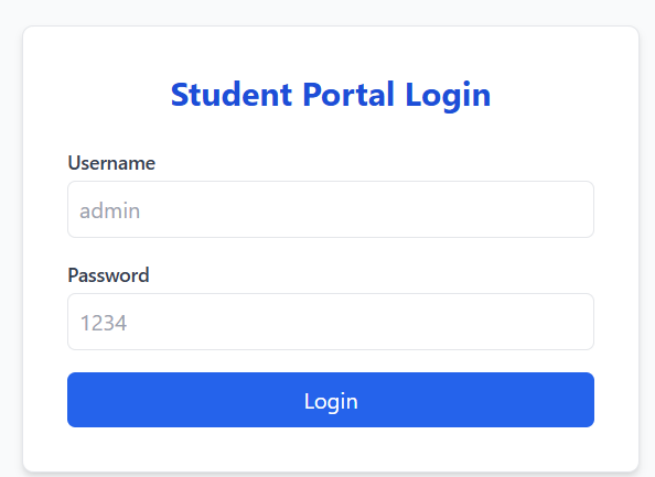
•

Step 6: Backend Verification & Database Querying ○ Create a server file (e.g., server.js) using Express.js. ○ Route Handling: Create a .post('/api/login') route. ○ Secure Retrieval: Extract the credentials from the request body. Query the Admin table in your database to find a matching username.

- Hash Comparison: Use the Bcrypt library's compare() function to check if the plain-text password submitted matches the hashed password stored in the database.

- Step 7: Token Generation & Session Management
    - Upon a successful match, use the jsonwebtoken library to sign a new JWT.
    - Send this token back to the browser in the JSON response.
    - Client-Side Storage: In script.js, receive the token and save it using localStorage.setItem('authToken', token).
    - Final Redirection: Use window.location.href = 'dashboard.html' to automatically move the authenticated user to the main system interface.

OUTPUT:



RESULT:

The login system was successfully transitioned from a static HTML page into a dynamic, secure authentication module. The implementation ensures that user data is validated on the client side, securely verified on the server side via password hashing, and maintained through a modern JWT-based session management system.

Exp.No:10 Date:	CI/CD Pipelines in Azure

AIM:

To automate the build-test-deploy lifecycle for the "Batch Import" module.

PROCEDURE:

Step 1: Source Control: Use the command git init, git add ., and git commit to prepare your code. Push the repository to your Azure Repo.

Step 2: Pipeline Initialization: Navigate to Pipelines in Azure DevOps. Click "New Pipeline" and select the "Azure Repos Git" source.

Step 3: YAML Configuration (CI): Select a "Node.js" or "Maven" starter template. Edit the azure-pipelines.yml to include:

- Task 1: npm install (to restore dependencies).
- Task 2: npm run test (to execute unit tests for the CSV parser).

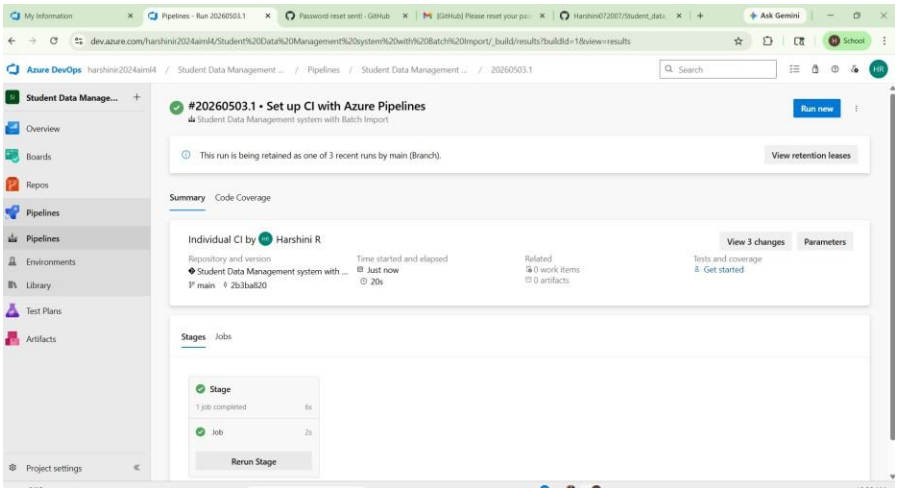
Step 4: Build Artifact: Add a task to "Archive Files" and "Publish Build Artifacts." This bundles your application into a deployable package.

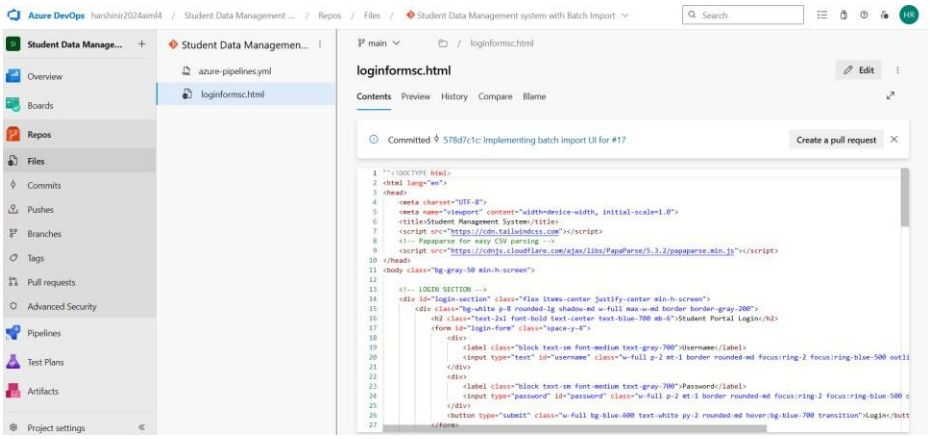
Step 5: Release Configuration (CD): Go to the Releases tab. Click "New Release Pipeline." Add the build artifact from Step 4 as the source trigger.

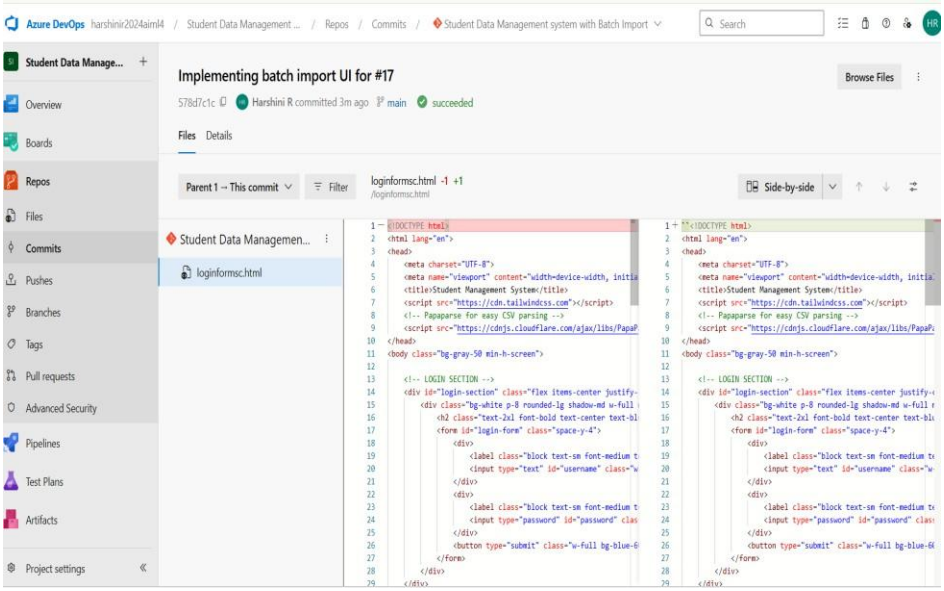
Step 6: Deployment Stage: Create a stage titled "Production." Add a task to deploy to an Azure App Service. Configure the "Web App Name" and "Service Connection."

Step 7: Automation Verification: Change a piece of code (e.g., update the login page title) and push it to the repo. Monitor the Pipeline Dashboard as it automatically builds, tests, and deploys the change to the cloud.

OUTPUT:







RESULT:

A fully automated DevOps pipeline was established, ensuring that every code update is verified through automated tests before reaching the production environment.